LAB 4: TIỀN XỬ LÝ DỮ LIỆU
(P1)

❖ MÔ TẢ BÀI LAB:

I – LÝ THUYẾT

- **Truy xuất Dataset**
- **Tìm dữ liệu còn thiếu**

I – THỰC HÀNH. (Thực hiện trên https://colab.research.google.com)

1. Khám phá hiệu suất của một số mô hình hồi quy tuyến tính trên tập dữ liệu trong thế giới thực, tức là MoneyBall. Xem mô tả về OpenML để biết thêm thông tin. Với bộ dữ liệu này, thu thập dữ liệu hiệu suất từ các cầu thủ bóng chày. Nhiệm vụ hồi quy là dự đoán chính xác số lần 'chạy' mà mỗi người chơi có thể ghi được và hiểu đâu là yếu tố quan trọng nhất.

```
if 'google.colab' in str(get_ipython()):
    !pip install openml

# General imports
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import openml as oml
import seaborn as sns
```

2. Nếu sử dụng Googe Colab dùng câu lệnh sau để cài đặt thư viên OpenML

```
if 'google.colab' in str(get_ipython()):
!pip install openml
```

```
# Download MoneyBall từ thư viên OpenML
moneyball = oml.datasets.get_dataset(41021)
# Tạo dataframe
X, y, _, attribute_names = moneyball.get_data(target=moneyball.default_target_attribute)
```

3. Đầu tiên, chúng ta hãy khám phá dữ liệu một cách trực quan bằng cách trực quan hóa phân phối giá trị và sự tương tác giữa mọi tính năng khác trong ma trận phân tán. Sử dụng tính năng mục tiêu làm biến màu để xem tính năng nào tương quan với mục tiêu. Để không hiện thị dư thừa, chúng ta cần xóa các tính năng phân loại (2 tính năng đầu tiên) và điền vào các giá trị còn thiếu. Hãy tìm xem những cột nào có giá trị bị thiếu.

```
#Tìm kiếm các giá trị còn thiếu với sklearn và xây dựng lại khung dữ liệu
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
X_clean_array = imputer.fit_transform(X[attribute_names[2:]]) # bỏ qua 2 tính năng đầu tiên
# Máy tính sẽ trả về một mảng có nhiều mảng. Để vẽ đồ thị, chúng tôi biến nó thành khung dữ một lần nữa.
X_clean = pd.DataFrame(X_clean_array, columns = attribute_names[2:]) #
```

```
pd.isnull(X).any()

Team          False
League        False
Year          False
RA            False
W             False
OBP           False
SLG           False
BA            False
Playoffs      False
```

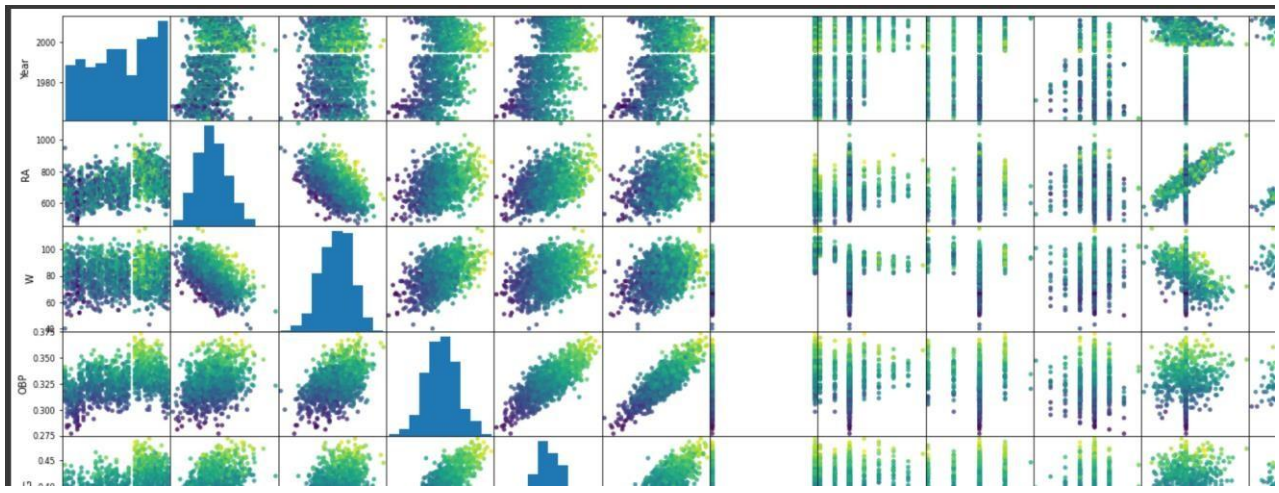
4. Để đạt được sự trực quan hóa tốt, chúng ta chỉ áp dụng các giá trị còn thiếu bằng cách sử dụng trung vị. Xóa tất cả các phiên bản có giá trị bị thiếu thực sự không phải là một tùy chọn vì một số tính năng có giá trị bị thiếu nhất quán: Kết quả sẽ phải xóa rất nhiều dữ liệu.

```
#Tìm kiếm các giá trị còn thiếu với sklearn và xây dựng lại khung dữ liệu
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(strategy="median")
X_clean_array = imputer.fit_transform(X[attribute_names[2:]]) # bỏ qua 2 tính năng đầu tiên
# Máy tính sẽ trả về một mảng có nhiều mảng. Để vẽ đồ thị, chúng tôi biến nó thành khung dữ một lần nữa.
X_clean = pd.DataFrame(X_clean_array, columns = attribute_names[2:]) #
```

5. Tiếp theo, chúng ta xây dựng ma trận phân tán, gồm cột mục tiêu để xem tính năng nào tương quan chặt chẽ với mục tiêu và cũng sử dụng giá trị mục tiêu làm màu để xem tổ hợp tính năng nào tương quan với mục tiêu.

```
# Ma trận phân tán của khung dữ liệu bao gồm tính năng đích
copyframe = X_clean.copy()
copyframe['y'] = pd.Series(y, index=copyframe.index)
scatter_matrix(copyframe, c=y, figsize=(25,25),
               marker='o', s=20, alpha=.8, cmap='viridis');
```

6. Xem kết quả từ việc tực quan hóa thông qua biểu đồ Scatter.



III – BÀI TẬP

1. Sử dụng dữ liệu Dataset tại:

https://drive.google.com/file/d/1iH0IU8ybFahoQTTf6A-Ipo6btOj44D1M/view?usp=sharing

2. Kiểm tra dữ liệu

```
df_netflix_2019.isnull().sum().sort_values(ascending=False)
```

```
df_netflix_2019.dtypes
```

```
director        1969
cast             570
country          476
date_added        11
rating            10
show_id            0
type               0
title              0
release_year       0
duration           0
listed_in          0
description        0
dtype: int64
```

1

```
show_id          int64
type            object
title           object
director        object
cast            object
country         object
date_added      object
release_year     int64
rating          object
duration        object
listed_in       object
description     object
dtype: object
```

2

3. Hiện thị các dữ liệu thiếu.

```
for column in df_netflix_2019.columns:
    percentage = df_netflix_2019[column].isnull().mean()
    print(f'{column}: {round(percentage*100, 2)}%')
```

```
show_id: 0.0%
type: 0.0%
title: 0.0%
director: 31.58%
cast: 9.14%
country: 7.64%
date_added: 0.18%
release_year: 0.0%
rating: 0.16%
duration: 0.0%
listed_in: 0.0%
description: 0.0%
```

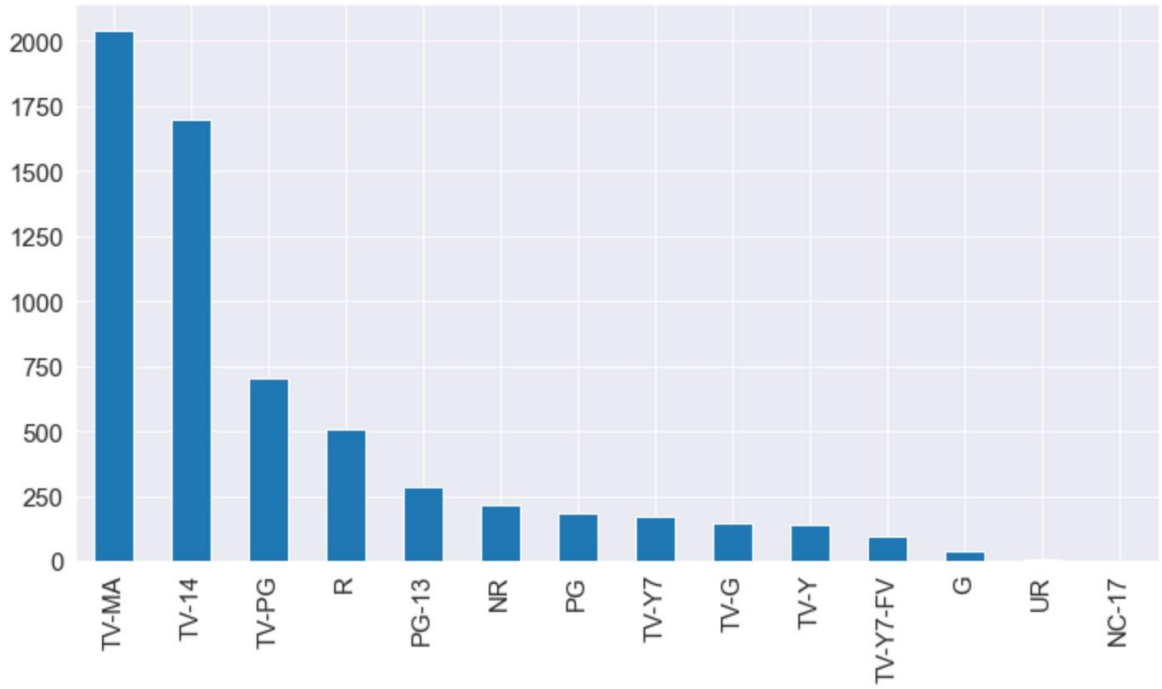
4. Xử lý missing data

```
#1. drop
#drop column
df_netflix_2019.drop('director', axis=1)
#drop row
no_director = df_netflix_2019[df_netflix_2019['director'].isnull()].index
df_netflix_2019.drop(no_director, axis=0)
#~ + .isnull()
df_netflix_2019[~df_netflix_2019['director'].isnull()]
#dropna()
df_netflix_2019.dropna(subset=['director'])

#2. use median, mean (numeric data) or mode (categorical data)
df_netflix_2019[df_netflix_2019['rating'].isnull()]
mode = ''.join(df_netflix_2019['rating'].mode())
df_netflix_2019['rating'].fillna(mode, inplace=True)
#3. replace with arbitrary number or text
df_netflix_2019['duration'].fillna(0, inplace=True)
```

4. Phân loại dữ liệu.

```
fig, ax = plt.subplots(nrows=1, ncols=1)
fig.set_size_inches(10, 6)
fig = df_netflix_2019['rating'].value_counts().plot.bar().get_figure()
plt.xticks(fontsize=15)
plt.yticks(fontsize=15)
fig.tight_layout()
fig.savefig('bar_cleaning')
```



==========